El caso de los Retrievers: teoría

Orquestación de flujos con LangChain y LlamaIndex

Elena Díaz - @HelenDiMo

Table of Contents

1. RAG en tres pasos

Un LLM no conoce tus documentos: ni los de tu empresa, ni los de tu expediente. **RAG** (*Retrieval-Augmented Generation*, generación aumentada con recuperación) lo soluciona en tres pasos:

1. **Indexar.** Se trocean los documentos y se guardan de forma que se puedan buscar.
2. **Recuperar.** Dada una pregunta, se buscan los trozos más relevantes. **Aquí trabaja el retriever.**
3. **Generar.** Se pasan esos trozos al LLM para que responda usándolos.

```
pregunta ──> RETRIEVER ──> pruebas ──> prompt ──> LLM ──> respuesta
              (busca)     (k trozos)
```

En esta píldora el caso es un crimen en Marlowe Hall, el expediente son 11 documentos cortos, el retriever es el ayudante de Holmes y el LLM es el propio Holmes.

2. ¿Qué es un retriever?

¿Qué es? Una interfaz muy simple: recibe un texto y devuelve una lista de documentos. No necesita guardar nada, solo saber dónde buscar. Por eso hay retrievers construidos sobre vector stores, sobre buscadores web, sobre bases de datos o sobre grafos.

¿Por qué importa? El LLM solo ve lo que le pasa el retriever. Si trae las pruebas equivocadas, el mejor modelo del mundo responde mal o se inventa cosas. Muchos fallos de un sistema RAG no son del modelo: son de la búsqueda.

¿Cómo se usa? Se crea a partir de un vector store (o de un índice) y se pide una lista de documentos. El mando principal es **k**: cuántos documentos quieres.

	LangChain	LlamaIndex
Crear	`vectorstore.as_retriever(search_kwargs={"k": 3})`	`index.as_retriever(similarity_top_k=3)`
Preguntar	`retriever.invoke(pregunta)`	`retriever.retrieve(pregunta)`

	LangChain	LlamaIndex
Devuelve	lista de `Document`	lista de `NodeWithScore` (con puntuación)
Encaja en una cadena	Sí: es un *Runnable* y se encadena con el operador pipe	A través de motores de consulta (`RetrieverQueryEngine`)

Un retriever propio puede ser tan pequeño como esto (LangChain):

```
from langchain_core.retrievers import BaseRetriever

class RetrieverMinimo(BaseRetriever):
    docs: list

    def _get_relevant_documents(self, query, *, run_manager):
        return [d for d in self.docs if query.lower() in
d.page_content.lower()]
```

3. Piezas previas

- **Documento / nodo.** Un texto con metadatos (título, autor, fecha). LangChain los llama `Document`; LlamaIndex, `Document` y `Node`.
- **Trozo (*chunk*).** Los documentos largos se trocean antes de indexarlos. En esta píldora cada documento es corto, así que cada uno es un único trozo.
- **Embedding.** Una lista de números que representa el **significado** de un texto. Dos textos que hablan de lo mismo tienen embeddings parecidos, aunque usen palabras distintas.
- **Vector store.** Donde se guardan los embeddings para poder compararlos rápido. Aquí se usa uno en memoria.
- **Similitud coseno.** La medida con la que se comparan dos embeddings (de -1 a 1; cuanto más alto, más parecidos).

4. Los cuatro retrievers

4.1 Similitud (por embeddings)

Qué hace. Convierte la pregunta en un embedding y trae los `k` documentos con embedding más parecido.

Cuándo. Cuando los usuarios preguntan con palabras distintas a las del documento. Con la pregunta «el sabueso» encuentra el testimonio del jardinero, que habla de un *lebrel* y de un *perro*; la palabra «sabueso» no aparece en ningún documento.

Su límite. Con palabras que el modelo no conoce, como el apellido «Whitcombe», las puntuaciones son casi ruido (alrededor de 0.1, frente a 0.46 de una búsqueda con sentido) y el orden no es fiable.

```
# LangChain
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```

```python
# LlamaIndex
retriever = index.as_retriever(similarity_top_k=3)
```

En LlamaIndex, `similarity_top_k` es un **máximo**, no una garantía: si hay filtros o pocos datos, pueden llegar menos resultados.

4.2 BM25 (palabras exactas)

Qué hace. Puntúa cada documento según cuántas palabras de la pregunta contiene, y da más valor a las palabras **raras** (las que aparecen en pocos documentos).

Cuándo. Con nombres propios, códigos, siglas y términos poco comunes. Con «Whitcombe» pone la carta al abogado la primera.

Su límite. No entiende sinónimos: con «el sabueso» no encuentra nada.

Trampa: el relleno. El BM25 de serie de LangChain **siempre devuelve k documentos**, aunque ninguno comparta ni una palabra con la pregunta (con puntuación 0). Si lo mezclas con otro retriever, esos documentos de relleno cuentan igual que los buenos. La solución es un BM25 *con corte*, que descarta las puntuaciones 0 (`src/sherlock/retrievers.py`).

```python
# LangChain
from langchain_community.retrievers import BM25Retriever
retriever = BM25Retriever.from_documents(docs, k=3)

# LlamaIndex (paquete aparte: llama-index-retrievers-bm25)
from llama_index.retrievers.bm25 import BM25Retriever
retriever = BM25Retriever.from_defaults(nodes=nodos,
similarity_top_k=3)
```

Trampa: los metadatos. El BM25 de LlamaIndex indexa también los metadatos (título, autor…) como si fueran texto. Si no quieres eso, exclúyelos con `excluded_embed_metadata_keys` al crear los `Document`.

4.3 MMR (variedad)

Qué hace. *Maximal Marginal Relevance*. Elige el primer resultado como la similitud, y para cada resultado siguiente penaliza los que se parecen a los ya elegidos.

Cuándo. Cuando tus resultados se repiten. En la pregunta «¿Quién vio luz en la biblioteca?» hay tres testimonios casi idénticos; con similitud se llevan los tres puestos, y con MMR entran el informe del forense y el diario de Watson.

Los mandos (LangChain). `lambda_mult` va de 0 (solo variedad) a 1 (solo relevancia; equivale a la similitud). `fetch_k` es cuántos candidatos mira MMR antes de elegir los `k` finales.

```python
# LangChain
retriever = vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 3, "fetch_k": 6, "lambda_mult": 0.5},
)
```

```
# LlamaIndex
retriever = index.as_retriever(
    similarity_top_k=3,
    vector_store_query_mode="mmr",
    vector_store_kwargs={"mmr_threshold": 0.6},
)
```

No son idénticos. El MMR de LlamaIndex es una versión simplificada: solo compara cada candidato con el **último** documento elegido, no con todos los anteriores como hace el de LangChain. Con un expediente pequeño se nota (puede colar documentos poco relevantes). Además, la puntuación que devuelve ya está ajustada por MMR: no es la similitud pura.

4.4 Híbrido (BM25 + similitud)

Qué hace. Combina varios retrievers. Hemos visto que BM25 y similitud fallan en sitios distintos, así que se usan los dos y se fusionan los resultados.

Cómo fusiona. No suma puntuaciones, porque las de BM25, las de similitud y las de cualquier otro método están en escalas distintas y no se pueden comparar. Usa **Reciprocal Rank Fusion** (RRF, fusión por rangos recíprocos): solo mira la **posición** de cada documento en cada lista.

> Cada lista da `peso / (posición + 60)` a cada documento, y se suman los puntos de todas las listas.

Ejemplo con «Whitcombe y el sabueso» y pesos 0.5 / 0.5:

Documento	En BM25	En similitud	Puntos
Carta de Lord Aldridge	puesto 1	no sale	0.5 / 61 = 0.0082
Testimonio del jardinero	no sale	puesto 1	0.5 / 61 = 0.0082
Diario de Watson	no sale	puesto 2	0.5 / 62 = 0.0081

Quedan arriba justo las dos pruebas que hacen falta: la carta (por la palabra rara) y el jardinero (por el significado).

```
# LangChain 1.x: ya NO está en langchain.retrievers
from langchain_classic.retrievers import EnsembleRetriever
retriever = EnsembleRetriever(retrievers=[bm25, similitud],
weights=[0.5, 0.5])

# LlamaIndex
from llama_index.core.retrievers import QueryFusionRetriever
retriever = QueryFusionRetriever(
    [similitud, bm25], similarity_top_k=3,
    num_queries=1,               # 1 = no reescribe la pregunta (no
necesita LLM)
    mode="reciprocal_rerank", use_async=False,
)
```

Detalles que despistan.

- El `EnsembleRetriever` devuelve la **unión** de las listas de cada retriever, no `k` documentos: hay que recortar (`[:3]`).
- Las puntuaciones del híbrido son puntos RRF (valores pequeños, como 0.017). No se pueden comparar con similitudes ni con puntuaciones BM25.
- Con `num_queries` mayor que 1, `QueryFusionRetriever` usa un LLM para reescribir la pregunta de varias formas y fusionar los resultados (*multi-query*).

5. La cadena completa

Los retrievers de LangChain son *Runnables*: piezas que se encadenan con el operador pipe (`|`), lo que se llama LCEL.

```
cadena = (
    {"contexto": retriever | formatear, "pregunta":
RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
respuesta = cadena.invoke("¿Qué prueba demuestra que quien envenenó
conocía la casa?")
```

Antes de culpar al LLM de una mala respuesta, **mira qué contexto recibió**: `(retriever | formatear).invoke(pregunta)`.

6. Errores típicos

Error	Cómo se nota	Arreglo
`k` mal elegido	Con `k` bajo falta la prueba clave; con `k` alto entra ruido	Medir con preguntas de prueba
BM25 «de relleno»	Devuelve `k` documentos con puntuación 0 y estropea el híbrido	Descartar puntuación 0
Culpar al LLM sin mirar el contexto	Respuestas raras	Imprimir lo que devolvió el retriever
Palabras desconocidas para el modelo	Puntuaciones bajas y orden arbitrario	Añadir BM25
Metadatos dentro del embedding (LlamaIndex)	Títulos y autores cuentan como texto	`excluded_embed_metadata_keys`
Imports antiguos (LangChain 1.x)	`ModuleNotFoundError` con `EnsembleRetriever`	`langchain_classic.retrievers`
Esperar que el híbrido recorte a `k`	Devuelve más de `k`	Cortar con `[:k]`
Comparar puntuaciones de métodos distintos	BM25, coseno y RRF están en escalas diferentes	Comparar posiciones

7. Cómo evaluar un retriever

Un retriever se evalúa **sin LLM**: preparas preguntas de prueba, anotas qué documentos deberían salir, y compruebas si salen entre los k primeros (*hit@k*). El notebook de solución lo hace con las cinco pistas del reto y obtiene:

	Similitud	BM25	MMR	Híbrido
Pistas resueltas (de 5)	1	2	2	4

Cómo leerlo: ningún retriever resuelve las cinco; el híbrido es el que más acierta, pero falla en la pista de los testigos repetidos, que solo resuelve MMR. Y cuidado: estas preguntas están **diseñadas como trampas** para cada método, así que no miden lo bueno que es cada uno en general. En un proyecto real hacen falta muchas más preguntas, sacadas de usuarios reales.

8. Glosario

Término	Significado
RAG	Generación aumentada con recuperación: buscar pruebas y dárselas al LLM
Retriever	Componente que recibe una pregunta y devuelve documentos
Embedding	Lista de números que representa el significado de un texto
k	Cuántos documentos devuelve el retriever
BM25	Búsqueda por palabras exactas que premia las palabras raras
MMR	Similitud que penaliza resultados repetidos
RRF	Fusión de listas que solo mira la posición de cada documento
Runnable	Pieza de LangChain que se puede encadenar con el operador pipe
LCEL	LangChain Expression Language: la forma de encadenar Runnables
Multi-query	Reformular la pregunta de varias formas (con un LLM) y fusionar resultados
HyDE	Un LLM convierte la pregunta en un documento hipotético que la responde, y se busca con él
Reranker	Modelo que reordena los resultados de un retriever

9. Qué se ha verificado y qué no

Todo el código del repositorio se ejecutó el 11 de octubre de 2026 con Python 3.13, `langchain-core` 1.6.9, `langchain-community` 0.4.2, `langchain-classic` 1.0.8, `llama-index-core` 0.14.25, `llama-index-retrievers-bm25` 0.8.0 y spaCy 3.8.16 (versiones exactas en `requirements-lock.txt`). Estas librerías cambian rápido: si algo no funciona, lo primero que hay que mirar son los nombres de los imports.

Verificado: los tres notebooks se ejecutan de principio a fin; los 17 tests de Python pasan; la web reproduce exactamente los resultados de los retrievers reales de LangChain (7 preguntas × 4 métodos) con `npm run check`.

No verificado:

- **Embeddings de Hugging Face** (`EMBEDDINGS_BACKEND=hf`): el entorno donde se construyó el repositorio no tenía acceso a Hugging Face. El código existe, pero no se ha probado.
- **LLM real** (Claude vía `langchain-anthropic`): no se probó con una clave. Sin clave se usa un «detective simulado» que solo enumera las pruebas.
- **Calidad de los embeddings por defecto.** Son vectores de palabras de spaCy (`es_core_news_md`) promediados por texto. Capturan sinónimos y temas, pero no el orden de las palabras. Con un modelo de embeddings distinto, los resultados y las puntuaciones cambiarán, y algunas «trampas» del corpus podrían comportarse de otra manera.

10. Recursos

- RAG From Scratch, lista de vídeos cortos de LangChain (en inglés) y sus notebooks. Son de 2024: los conceptos siguen valiendo, pero los imports pueden haber cambiado.
- Curso completo en freeCodeCamp (unas 2,5 horas, en inglés)
- Conceptos de retrievers en LangChain (versión JavaScript; los conceptos son los mismos que en Python)
- Módulos de retrievers de LlamaIndex
- BM25 Retriever en LlamaIndex
- Paper de Reciprocal Rank Fusion